```
raise

raise "Missing name" if name.nil?

if i >= names.size
  raise IndexError, "#{i} >= size (#{names.size})"
end

raise ArgumentError, "Name too big", caller
```

In the last example, we show that we can add a stack trace as an optional third argument to the raise method. The effect of this usage is to remove the current routine from the stack backtrace, which is often useful in library modules. We do this using the Kernel#caller method, which returns the current stack trace. The ability to edit the stack trace sent to the exception for more focused information is why you have the optional third argument.

The Kernel#caller method just returns an array of strings with information about the call stack. We can take this further by just manipulating that array. The following code removes two routines from the backtrace by passing only a subset of the call stack to the new exception:

```
raise ArgumentError, "Name too big", caller[1..]
```

Adding Information to Exceptions

You can define your own exceptions to hold any information that you need to pass out from the site of an error. For example, certain types of network errors may be transient depending on the circumstances. If such an error occurs and the circumstances are right, you could set a flag in the exception to tell the handler that it may be worth retrying the operation.

Here's what part of the remote server from the previous example might look like.

tut_exceptions/retry_exception.rb

```
class RetryException < RuntimeError
  attr_reader :ok_to_retry
  def initialize(ok_to_retry)
    @ok_to_retry = ok_to_retry
  end
end
```

Somewhere down in the depths of the code, a transient error occurs:

tut_exceptions/read_data.rb

```
def read_data(attempt_count)
  data = @socket.read(512)
  if data.nil?
    raise RetryException.new(attempt_count < 10), "transient read error"
  end
  # .. normal processing
end
```

And we might incorporate that in our call:

```
attempts = 0
begin
  attempts += 1
  @connection = @remote_server.read_data(attempts)

rescue RetryException => e
```